

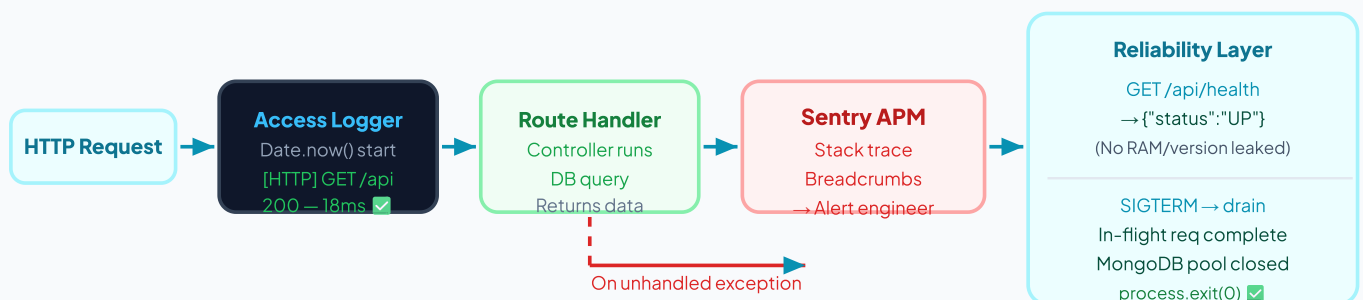
Observability, Monitoring & Reliability

Sentry APM real-time error tracking, HTTP access latency telemetry, sanitized liveness health endpoint, graceful SIGTERM shutdown, and RPO/RTO disaster recovery targets.

🔥 APM: Sentry Error Tracking ❤️ Health: /api/health Probe 🛡️ DR: RPO <1hr · RTO <15min

1. Observability & Reliability Architecture

TELEMETRY STACK — ERROR CAPTURE, LATENCY LOGGING & HEALTH PROBING



DISASTER RECOVERY ARCHITECTURE — RPO & RTO TARGETS

RPO < 1 Hour

Recovery Point Objective — Max Data Loss Window

- ✓ MongoDB Atlas continuous oplog snapshots
 - ✓ Point-in-time restore to any second in last 24h
 - ✓ 3-node replica set — data survives single node failure
- **Maximum data loss: 1 hour of new submissions**

RTO < 15 Minutes

Recovery Time Objective — Max Downtime Window

- ⚡ Vercel rollback: <10 seconds (DNS pointer swap)
 - ⚡ Render new container spin-up: 2-5 minutes
 - ⚡ MongoDB replica failover: <30 seconds auto
- **Max downtime: 15 minutes end-to-end**

```
// server.js — Graceful Shutdown Handler (SIGTERM from cloud container orchestrator)
const shutdown = async (signal) => { console.log(`[PROCESS] ${signal} received — initiating graceful drain...`); server.close(async () => { console.log(`[PROCESS] HTTP server closed. Closing MongoDB pool...`); await mongoose.connection.close(false); // false = don't force close console.log(`[PROCESS] All connections closed safely. process.exit(0)`); process.exit(0); }); }; process.on('SIGTERM', () => shutdown('SIGTERM')); // Cloud PaaS deploy signal
process.on('SIGINT', () => shutdown('SIGINT')); // Ctrl+C local dev signal
```

2. Technology Decision Framework

1. Sentry APM vs Raw Cloud Logs vs Datadog

OBSERVABILITY

WHAT IS IT?

Sentry intercepts every unhandled Node.js exception and React component error, grouping them with full stack traces, user breadcrumbs, and environment context.

HOW DOES IT WORK & PROBLEM SOLVED?

Without APM, a student who encounters a 500 error sees a generic failure message and you only discover the bug if they contact support. Sentry captures the error 100ms after it occurs, pushes an alert, and shows the exact line of code and preceding user actions that caused it.

WHAT ALTERNATIVES EXISTED?

Raw Render/CloudWatch Logs: Errors buried in thousands of lines, no grouping, no alert thresholds. **Datadog:** Enterprise pricing (\$150–\$500/month) with complex agent installation.

WHY DID WE CHOOSE THIS APPROACH?

Free tier covers 5,000 errors/month — sufficient for early-stage. Zero-config Node.js SDK integration. Source map uploads de-minify production stack traces to exact original source lines.

2. Sanitized Health Endpoint vs Verbose Debug Probe

SECURITY

WHAT IS IT?

A GET /api/health endpoint that returns only {"status": "UP", "timestamp": ...} in production, withholding process memory stats and runtime version details.

HOW DOES IT WORK & PROBLEM SOLVED?

Many tutorials expose process.memoryUsage() and process.version in health endpoints. Attackers use automated scanners to discover Node.js version numbers and target known CVEs for that specific runtime version. Our sanitized response provides liveness confirmation while revealing zero infrastructure metadata.

WHAT ALTERNATIVES EXISTED?

Verbose /health endpoints exposing Node version, heap usage, uptime — providing attackers a fingerprinting vector for targeted CVE exploitation.

WHY DID WE CHOOSE THIS APPROACH?

Satisfies PaaS liveness probe requirements while adhering to OWASP information hiding principles.

💡 Senior Interview Defense: "How do you ensure system reliability and observability?"

"UniCoach has a 4-pillar reliability stack. Sentry captures and alerts on every unhandled exception with breadcrumbs showing the exact user action sequence that triggered it. A lightweight access logger records every route's HTTP status and execution latency in milliseconds for trend analysis. The /api/health endpoint is deliberately sanitized — it returns only status:UP with no runtime metadata, preventing Node.js version fingerprinting by attackers. Finally, SIGTERM handlers drain in-flight requests before closing MongoDB connection pools — preventing any data corruption during rolling cloud deployments. Our RPO is under 1 hour via Atlas oplog snapshots and RTO under 15 minutes via instant Vercel rollbacks."